

Proyecto Final de Compilación

Analizador para el lenguaje RubricaLang

Loren Odette Paz Lavilla

Número de estudiante: 23

Descripción del DSL y su propósito

RubricaLang es un lenguaje de dominio específico (DSL) diseñado para definir rúbricas académicas de manera estructurada, legible y verificable automáticamente. Permite declarar criterios de evaluación, asignar pesos porcentuales a cada criterio y definir niveles de desempeño con puntuaciones concretas, todo ello dentro de una sintaxis clara inspirada en los lenguajes de programación tradicionales. El propósito principal del lenguaje es validar de forma estática tres reglas semánticas esenciales que garantizan la corrección de la rúbrica: en primer lugar, que la suma de los pesos de todos los criterios sea exactamente 100, asegurando que la ponderación total sea completa; en segundo lugar, que ningún peso sea negativo ni cero, pues todo criterio debe contribuir positivamente a la nota final; y en tercer lugar, que cada criterio tenga al menos un nivel de desempeño, evitando criterios vacíos que no puedan ser evaluados. De esta manera, RubricaLang automatiza la detección de errores comunes que suelen pasar desapercibidos cuando se crean rúbricas manualmente con procesadores de texto u hojas de cálculo. Adicionalmente, el lenguaje incorpora características de propósito general que lo convierten en una herramienta más potente y versátil: incluye declaración de variables con tipos básicos (números enteros, decimales y cadenas), asignaciones, expresiones aritméticas y relacionales con precedencia de operadores, y estructuras de control como el condicional si (con opción sino) y el bucle mientras. Gracias a estas adiciones, el usuario no solo puede definir la rúbrica, sino también automatizar cálculos auxiliares, como la comprobación de la suma de pesos mediante una expresión aritmética o la generación condicional de mensajes de validación, todo dentro del mismo lenguaje. En conjunto, RubricaLang ofrece un entorno seguro y expresivo para la creación de instrumentos de evaluación académica, reduciendo la carga cognitiva del docente y aumentando la fiabilidad de las rúbricas.

Tabla completa de tokens (expresiones regulares)

Token	Expresión regular	Descripción
RUBRICA	rubrica	Palabra reservada para iniciar una rúbrica
CRITERIO	criterio	Palabra reservada para definir un criterio
PESO	peso	Palabra reservada para indicar el peso
NIVEL	nivel	Palabra reservada para definir un nivel
TOTAL	total	Reservada (para extensión)
DESCRIPCION	descripcion	Reservada (para extensión)
SI	si	Palabra reservada para condicional

Token	Expresión regular	Descripción
SINO	sino	Palabra reservada para else
MIENTRAS	mientras	Palabra reservada para bucle
TIPO_NUMERO	numero	Tipo de dato entero
TIPO_DECIMAL	decimal	Tipo de dato real
TIPO_CADENA	cadena	Tipo de dato cadena
IDENT	[a-zA-Z_][a-zA-Z0-9_]*	Identificador de variable o nombre
NUMERO	\d+	Literal entero
DECIMAL	\d+\.\d+	Literal decimal
CADENA	\"([^\n] (\\.))*?\"	Literal de cadena con escapes \n, \t, \", \\
MAS	+	Suma
MENOS	-	Resta o unario
MULT	*	Multiplicación
DIV	/	División
MENOR	<	Menor que
MAYOR	>	Mayor que
MENOR_IGUAL	<=	Menor o igual
MAYOR_IGUAL	>=	Mayor o igual
IGUAL	==	Igualdad
DISTINTO	!=	Diferente
ASIGN	=	Asignación
PAR_IZQ	(	Paréntesis izquierdo
PAR_DER	)	Paréntesis derecho
LLAVE_IZQ	{	Llave izquierda

Token	Expresión regular	Descripción
LLAVE_DER	}	Llave derecha
PUNTO_Y_COMA	;	Fin de sentencia
COMA	,	Separador
DOS_PUNTOS	:	(No usado, pero definido)
Comentarios	//[^\\n]* (línea) y /*[\\s\\S]*?*/ (bloque)	Se ignoran

Gramática formal en EBNF

(* Programa *)

programa = { sentencia } ;

(* Sentencias generales *)

sentencia = declaracion_var | asignacion | estructura_control | definicion_rubrica ;

(* Declaraciones con o sin inicialización *)

declaracion_var = tipo , IDENT , ["=" , expresion] , ";" ;

tipo = TIPO_NUMERO | TIPO_DECIMAL | TIPO_CADENA ;

asignacion = IDENT , "=", expresion , ";" ;

(* Estructuras de control *)

estructura_control = "si" , "(" , expresion , ")" , bloque , ["sino" , bloque]
| "mientras" , "(" , expresion , ")" , bloque ;

bloque = "{" , { sentencia } , "}" ;

(* Construcción específica del dominio *)

definicion_rubrica = "rubrica" , IDENT , "{" , { definicion_criterio } , "}" ;

definicion_criterio = "criterio" , IDENT , "peso" , expresion , "{" , { definicion_nivel } , "}" ,
";" ;

definicion_nivel = "nivel" , CADENA , expresion , ";" ;

(* Expresiones con precedencia (niveles) *)

expresion = expresion_logica ;

expresion_logica = expresion_relacional , { ("==" | "!=") , expresion_relacional } ;

expresion_relacional = expresion_aditiva , { ("<" | ">" | "<=" | ">=") , expresion_aditiva } ;

expresion_aditiva = expresion_multiplicativa , { ("+" | "-") , expresion_multiplicativa } ;

expresion_multiplicativa = expresion_unaria , { ("*" | "/") , expresion_unaria } ;

expresion_unaria = ["+" | "-"] , primaria ;

primaria = NUMERO | DECIMAL | CADENA | IDENT | "(" , expresion , ")" ;

Precedencia de operadores (de menor a mayor):

1. ==, != (igualdad)
2. <, >, <=, >= (relacionales)
3. +, - (aditivos)
4. *, / (multiplicativos)

5. - unario (mayor prioridad)

Reglas semánticas explicadas con ejemplos

Reglas generales :

Variable no declarada: si se usa un identificador que no ha sido declarado en ningún ámbito accesible.

Ejemplo: $x = 5$; sin un numero x ; previo $\rightarrow$ error.

Redeclaración en el mismo ámbito: no se puede declarar dos veces la misma variable.

Ejemplo: numero a ; numero a ; $\rightarrow$ error.

Tipo incompatible en asignación o expresión: no se puede asignar un valor de tipo cadena a una variable numero.

Ejemplo: numero $x = \text{"hola"}$; $\rightarrow$ error.

Uso de variable antes de inicialización: leer una variable que nunca recibió un valor.

Ejemplo: numero y ; $z = y + 1$; $\rightarrow$ error (y no inicializada).

Reglas propias del dominio (RubricaLang):

Suma de pesos igual a 100: al final de cada rúbrica, se suman los pesos de todos sus criterios. Si no es exactamente 100, se reporta error.

Ejemplo: criterio A peso 40 {...}; criterio B peso 50 {...} $\rightarrow$ suma 90 $\rightarrow$ error.

Peso mayor que cero: cada peso debe ser un número positivo. Peso cero o negativo no está permitido.

Ejemplo: criterio X peso 0 {...} $\rightarrow$ error.

Cada criterio debe tener al menos un nivel: un criterio sin niveles no define desempeño.

Ejemplo: criterio Y peso 30 { } $\rightarrow$ error.

Además, se verifica que los pesos y las puntuaciones de los niveles sean **constantes numéricas** (no expresiones con variables), para facilitar la validación estática.

Decisiones de diseño y justificación de la herramienta elegida

Herramienta: Python con la biblioteca PLY (Python Lex-Yacc).

Justificación: PLY es de uso libre, se integra fácilmente con Python, permite construir un AST manualmente y tiene buena documentación. Además, Python es un lenguaje de alto nivel que facilita la depuración y el manejo de errores. No se usaron generadores como Flex/Bison porque se buscaba un prototipo rápido y legible para un DSL pequeño.

Estructura del AST: Se definió una jerarquía de clases (herencia desde ASTNode) para cada construcción del lenguaje. Esto facilita el recorrido posterior durante el análisis semántico (patrón visitor implementado manualmente). Cada nodo almacena información de línea y columna para reportar errores precisos.

Tabla de símbolos: Se implementó con una pila de diccionarios para manejar ámbitos anidados (bloques {...} y rúbricas). Cada símbolo guarda su tipo y si ha sido inicializado.

Recuperación de errores sintácticos: Se adoptó el modo pánico: cuando se encuentra un token inesperado, el parser avanza hasta encontrar un token de sincronización (;, }, rubrica, si, mientras). Esto evita múltiples errores falsos y permite continuar analizando el resto del programa.

Análisis semántico de constantes: Se implementó una función evaluar_constante que recorre el árbol de una expresión y calcula su valor si todas las hojas son literales numéricos; de lo contrario retorna None. Esto permite verificar que los pesos y puntuaciones de la rúbrica sean valores fijos, no dependientes de variables.

Instrucciones para compilar y ejecutar

Requisitos previos:

Python 3.6 o superior instalado.
Instalar PLY: ejecutar en terminal pip install ply.

Pasos para ejecutar el analizador:

Descargar o clonar el repositorio con todos los archivos (ast.py, lexer.py, parser.py, semantic.py, main.py).

Colocar el programa fuente (ej. valida.rub) en el mismo directorio.
Abrir una terminal en ese directorio y ejecutar:

```
python main.py valida.rub
```

El analizador mostrará los resultados del análisis léxico, sintáctico y semántico en la consola.

Programas de prueba

Programa válido (valida.rub)

```
// Programa válido: suma de pesos 100, todos >0, niveles presentes
numero x = 10;
cadena nombre = "Rubrica Final";
```

```
rubrica examen_final {
    criterio teoria peso 40 {
        nivel "excelente" 10;
        nivel "bueno" 7;
        nivel "regular" 4;
    };
    criterio practica peso 60 {
        nivel "excelente" 10;
        nivel "bueno" 7;
    };
}
```

```
x = x + 5;
```

Programa con error sintáctico (error_sintactico.rub)

```
rubrica examen {
    criterio teoria peso 40 // falta la llave '{'
        nivel "excelente" 10;
    };
};
```

Programa con error semántico (error_semantico.rub)

```
rubrica examen {  
    criterio teoria peso 40 { nivel "bueno" 7; };  
    criterio practica peso 50 { nivel "bueno" 7; };  
}  
// suma = 90, debería ser 100
```

Ejemplo de salida para un programa válido:

=== ANÁLISIS LÉXICO ===

No se encontraron errores léxicos.

=== ANÁLISIS SINTÁCTICO ===

El análisis sintáctico se completó con éxito. AST construido.

=== ANÁLISIS SEMÁNTICO ===

No se encontraron errores semánticos. El programa es válido.

Ejemplo de salida para un programa con error sintáctico

=== ANÁLISIS LÉXICO ===

No se encontraron errores léxicos.

=== ANÁLISIS SINTÁCTICO ===

Error sintáctico [línea 3, columna 10]: token inesperado 'nivel'

El programa contiene errores sintácticos. No se procede al análisis semántico.

Ejemplo de salida para un programa con error semántico

=== ANÁLISIS LÉXICO ===

No se encontraron errores léxicos.

=== ANÁLISIS SINTÁCTICO ===

El análisis sintáctico se completó con éxito. AST construido.

=== ANÁLISIS SEMÁNTICO ===

Error semántico [línea 1, columna 9]: la suma de pesos de los criterios es 90, debe ser 100

[ERROR] El programa contiene errores semánticos.